

Adam Optimizer From Scratch

A simple educational implementation of the **Adam (Adaptive Moment Estimation) optimizer** from scratch using Python.

The main goal of this project is to understand the mathematical and algorithmic structure of Adam instead of treating it as a black-box optimizer.

What is Adam?

Adam is an optimization algorithm used to update model parameters during machine learning and deep learning training.

Adam combines information from:

- The first moment of gradients
- The second moment of gradients
- Bias correction

This allows Adam to adapt the update step based on the behavior of the gradients.

Adam Equations

First Moment

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Second Moment

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Bias Correction

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Parameter Update

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Default Adam Parameters

```
Learning Rate = 0.001
Beta 1        = 0.9
```

Beta 2	= 0.999
Epsilon	= 1e-8

Project

This project uses a simple linear regression model:

$$\hat{y} = wx + b$$

The synthetic dataset is generated from:

$$y = 3x + 5 + noise$$

The model is trained using the Adam optimization algorithm.

Dataset Split

The dataset is divided into three parts:

Training	70%
Validation	15%
Testing	15%

Training Set

Used to calculate gradients and update the model parameters.

Validation Set

Used during training to monitor how well the model generalizes to unseen data.

Test Set

Used only after training for the final evaluation.

Loss Function

Mean Squared Error (MSE) is used:

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Implementation

The Adam optimizer is implemented manually.

The implementation includes:

1. Gradient calculation
2. First moment estimation
3. Second moment estimation
4. Bias correction
5. Adaptive parameter updates
6. Training loss calculation
7. Validation loss calculation
8. Final test evaluation

No machine learning framework is used for the optimizer implementation.

Libraries

The core implementation uses only Python's built-in `math` and `random` modules.

`Matplotlib` is used for visualization.

No PyTorch, TensorFlow, or Scikit-learn is required.

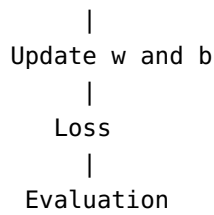
Visualization

The project produces visualizations for:

- Training data
- Validation data
- Test data
- Final model prediction
- Training loss
- Validation loss
- Final test loss

Learning Flow

```
Dataset
|
+---- Train
|
+---- Validation
|
+---- Test
      |
      |
      Linear Model
      |
      Gradient
      |
      Adam
```



Why implement Adam from scratch?

Modern frameworks provide Adam as a ready-to-use optimizer, for example:

```
optimizer = Adam(...)
```

However, implementing it manually makes the internal mechanism easier to understand.

This project focuses on understanding what happens between:

```
Gradient
  ↓
Adam
  ↓
Parameter Update
```

instead of treating the optimizer as a black box.

Future Improvements

Possible improvements include:

- Implement Adam for multiple parameters
- Implement Mini-Batch Adam
- Compare Adam with SGD
- Compare Adam with Momentum
- Implement AdamW
- Train a small neural network using the optimizer
- Implement the optimizer using NumPy
- Compare the custom implementation with PyTorch Adam
- Add early stopping
- Add learning-rate experiments

Technologies

- Python
- Math
- Random
- Matplotlib

Author

Mosa

Computer Science Student | Machine Learning Enthusiast